

Final Project: Evaluating Public Trust in Newspaper Graphics

STAT 158

Clara Reckhorn, Jocelyne Perez, Gabriella Rachal
University of California, Berkeley

Index

1	Introduction	2
2	Methodology	2
3	Results	2
4	Discussion or Conclusion	2
5	Appendix	2
5.1	Use of AI	2
5.2	Significant Changes Made After Protocol	2
5.3	Lab Notes	2
5.4	Code	3

1 Introduction

2 Methodology

3 Results

4 Discussion or Conclusion

Based on our calculations, the factors that are statistically significant are (in order from most to least significant): source credibility ($p = 0.0006714$), headline framing (0.0014075), and color (0.0240717). The block ($p = 0.82$) and all two-way and three-way interactions are far from significance (all $p > 0.25$), which suggests that the three factors act independently with no meaningful interaction effects.

However, for color, our power is only 0.429, which indicates that if a true effect exists, we only have about a 43 percent chance of detecting it. This result should be interpreted cautiously and it would have been mostly uninformative also if it had resulted as non-significant.

Additionally, our two- and three-way interaction effects do not have a high enough power to be revealing. To increase power for these estimates of the effects, we would need a larger sample size.

5 Appendix

5.1 Use of AI

5.2 Significant Changes Made After Protocol

We needed to change the Google Form because it was sending users back to the main page because of how we had directed the navigation. This meant we had to delete the first 7 entries and collect a bit more data than we had expected, but ultimately we still received 50 user entries.

5.3 Lab Notes

```
library(tidyverse)
```

```
#####  
# Reading in and formatting raw lab data  
#####  
  
data <- read_csv("data/raw-data.csv")  
  
data
```

```
# A tibble: 50 × 21  
  `Unit ID` Timestamp What is your political ...1 Did you get number  
1...2
```

```

      <dbl> <chr>           <chr>
<chr>
1          1 4/22/2026 13:49:38 Middle
<NA>
2          2 4/22/2026 13:50:05 Middle
no
3          3 4/22/2026 13:50:29 Left
<NA>
4          4 4/22/2026 13:51:11 Left
<NA>
5          5 4/22/2026 13:51:49 Left
<NA>
6          6 4/22/2026 13:52:24 Left
<NA>
7          7 4/22/2026 13:52:31 Middle
yes
8          8 4/22/2026 13:53:01 Right
no
9          9 4/22/2026 13:53:17 Middle
<NA>
10         10 4/22/2026 13:53:40 Left
<NA>
# i 40 more rows
# i abbreviated names: 1`What is your political affiliation?`,
#   2`Did you get number 1?`
# i 17 more variables: `Did you get number 2?` <chr>,
#   `Did you get number 3?` <chr>, `Did you get number 4?` <chr>,
#   `Did you get number 5?` <chr>, `Did you get number 6?` <chr>,
#   `Did you get number 7?` <chr>, `Did you get number 8?` <chr>, ...

```

5.4 Code

```

#####
# Reading in and formatting data
#####

data <- read_csv("data/data.csv")

data

```

```

# A tibble: 50 × 13
      id   xi  x1  x2  x3  x4  x5  x6  x7  x8  di
  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1     1     4  NA  NA  NA   1  NA  NA  NA  NA   6
2     2     8  NA  NA  NA  NA  NA  NA  NA   1   4
3     3     7  NA  NA  NA  NA  NA  NA   1  NA   4
4     4     7  NA  NA  NA  NA  NA  NA   1  NA   6

```

```

5      5      5      NA      NA      NA      NA      1      NA      NA      NA      3
6      6      2      NA      1      NA      NA      NA      NA      NA      NA      5
7      7      1      1      NA      NA      NA      NA      NA      NA      NA      4
8      8      7      NA      NA      NA      NA      NA      NA      1      NA      6
9      9      7      NA      NA      NA      NA      NA      NA      1      NA      1
10     10     6      NA      NA      NA      NA      NA      1      NA      NA      3
# i 40 more rows
# i 2 more variables: `political aff` <chr>, block <dbl>

```

```

power_data <- data |>
  mutate(
    Y = di,
    treatment = factor(xi),
    block = factor(block)
  ) |>
  select(treatment, Y, block)

```

```
power_data
```

```

# A tibble: 50 × 3
  treatment      Y block
  <fct>      <dbl> <fct>
1 4          6 2
2 8          4 2
3 7          4 1
4 7          6 1
5 5          3 1
6 2          5 1
7 1          4 2
8 7          6 3
9 7          1 2
10 6         3 1
# i 40 more rows

```

```

treatment_map <- data.frame(
  treatment      = factor(1:8),
  source_credibility =
factor(c("high", "high", "high", "high", "low", "low", "low", "low")),
  color          = factor(c("yes", "yes", "no", "no", "yes", "yes", "no", "no")),
  headline_framing =
factor(c("sensationalist", "neutral", "sensationalist", "neutral",
"sensationalist", "neutral", "sensationalist", "neutral"))
)

```

```
# Joining the power_data back in to match it to treatment conditions so we can
do 2^3 factorial design
power_data_full <- power_data |>
  left_join(treatment_map, by = "treatment") |>
  select(source_credibility, color, headline_framing, block, Y)

power_data_full
```

```
# A tibble: 50 × 5
  source_credibility color headline_framing block      Y
  <fct>              <fct> <fct>          <fct> <dbl>
1 high              no    neutral         2      6
2 low              no    neutral         2      4
3 low              no    sensationalist  1      4
4 low              no    sensationalist  1      6
5 low              yes    sensationalist  1      3
6 high              yes    neutral         1      5
7 high              yes    sensationalist  2      4
8 low              no    sensationalist  3      6
9 low              no    sensationalist  2      1
10 low             yes    neutral         1      3
# i 40 more rows
```

```
#####
# Complete Block Design
#####

# political affiliation level is the block
# each participant sees exactly 1 graph (1 treatment combination)

# source cred: 0 = low, 1 = high
# color: 0 = no, 1 = yes
# headline framing: 0 = neutral, 1 = sensationalist

#####
# Step 1: Linear model and the data generating process
#####

dry_run <- data.frame(
  unit      = 1:16,
  block     = rep(c("democrat", "republican"), each = 8),
  source_credibility = rep(c("high", "high", "high", "high",
                             "low", "low", "low", "low"), 2),
  color     = rep(c("yes", "yes", "no", "no",
                    "yes", "yes", "no", "no"), 2),
```

```

headline_framing = rep(c("sensationalist", "neutral",
                          "sensationalist", "neutral",
                          "sensationalist", "neutral",
                          "sensationalist", "neutral"), 2),
response          = c(6, 7, 5, 5, 4, 4, 3, 4, # democrat block
                      5, 6, 4, 3, 5, 3, 2, 4) # republican block
)

```

```

#####
# Writing out the linear model as an R function
#####

set.seed(42)

generate_Y <- function(mu, alpha, gamma, delta, beta,
                      alpha_gamma, alpha_delta, gamma_delta,
                      alpha_gamma_delta,
                      sigma_sq, j, k, l, i, m) {

  n <- length(m)

  epsilon <- rnorm(n, mean = 0, sd = sqrt(sigma_sq))

  ag <- mapply(function(jj, kk) alpha_gamma[jj, kk], j, k)
  ad <- mapply(function(jj, ll) alpha_delta[jj, ll], j, l)
  gd <- mapply(function(kk, ll) gamma_delta[kk, ll], k, l)
  agd <- mapply(function(jj, kk, ll) alpha_gamma_delta[jj, kk, ll], j, k, l)

  Y <- mu + beta[i] + alpha[j] + gamma[k] + delta[l] + ag + ad + gd + agd +
  epsilon

  return(Y)
}

```

```

#####
# Step 2: Choosing parameters
#####

# Defining parameter values

mu <- 0

alpha <- c(-0.55, 0.55) # source credibility: ~1.5 point difference
gamma <- c(-0.45, 0.45) # color: ~0.9 point difference
delta <- c(-0.55, 0.55) # headline framing: ~1.1 point difference

```

```

beta <- c(-0.10, 0.10) # block

mu <- mean(power_data_full$Y)

sigma_sq <- var(power_data_full$Y)
n <- 16

alpha_gamma <- matrix(c(0.15, -0.15, -0.15, 0.15), nrow = 2, ncol = 2)
alpha_delta <- matrix(c(0.10, -0.10, -0.10, 0.10), nrow = 2, ncol = 2)
gamma_delta <- matrix(c(0.08, -0.08, -0.08, 0.08), nrow = 2, ncol = 2)
alpha_gamma_delta <- array(c(0.05, -0.05, -0.05, 0.05,
                             -0.05, 0.05, 0.05, -0.05), dim = c(2, 2, 2))

```

```

#####
# Estimate for sigma squared
#####

sigma_sq

```

```
[1] 2.275918
```

```

#####
# Step 3: Simulating Inference
#####

n_per_cell <- 3
i <- as.integer(power_data_full$block)
j <- as.integer(power_data_full$source_credibility) # high=1, low=2
k <- as.integer(power_data_full$color) # no=1, yes=2
l <- as.integer(power_data_full$headline_framing) # neutral=1,
sensationalist=2
m <- 1:nrow(power_data_full) # n_total = 50

n_total <- length(m)

Y <- generate_Y(mu, alpha, gamma, delta, beta,
               alpha_gamma, alpha_delta, gamma_delta, alpha_gamma_delta,
               sigma_sq, j, k, l, i, m)

dat <- data.frame(
  Y = Y,
  block = factor(i),
  source_credibility = factor(j),
  color = factor(k),

```

```

headline_framing = factor(l)
)

fit <- lm(Y ~ source_credibility * color * headline_framing + block, data =
dat)
aov_tab <- anova(fit)

#####
# ANOVA (Table 2 & 3 in text)
#####

aov_tab

```

Analysis of Variance Table

Response: Y

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
source_credibility	1	57.719	57.719	22.0933	3.74e-05
color	1	12.002	12.002	4.5941	0.03891
headline_framing	1	19.041	19.041	7.2882	0.01051
block	1	0.314	0.314	0.1202	0.73080
source_credibility:color	1	1.421	1.421	0.5438	0.46565
source_credibility:headline_framing	1	0.425	0.425	0.1627	0.68905
color:headline_framing	1	0.635	0.635	0.2431	0.62499
source_credibility:color:headline_framing	1	10.198	10.198	3.9034	0.05589
Residuals	36	94.050	2.613		

```

source_credibility ***
color *
headline_framing *
block
source_credibility:color
source_credibility:headline_framing
color:headline_framing
source_credibility:color:headline_framing .
Residuals
---
```

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

#####
# Fit linear model
#####

fit <- lm(Y ~ source_credibility * color * headline_framing + block, data =
dat)

```



```
#####
# Extract ANOVA table
#####

aov_tab <- anova(fit)

#####
# Get p-value for source credibility (main effect of interest)
#####

p_val <- aov_tab["source_credibility", "Pr(>F)"]

#####
# Convert to rejection indicator
#####

reject <- as.integer(p_val < 0.05)
reject
```

```
[1] 1
```

```
#####
# Step 4: Iterating to calculate Power
#####

one_sim <- function(mu, alpha, gamma, delta, beta,
                    alpha_gamma, alpha_delta, gamma_delta, alpha_gamma_delta,
                    sigma_sq, i, j, k, l, m) {

  Y <- generate_Y(mu, alpha, gamma, delta, beta,
                 alpha_gamma, alpha_delta, gamma_delta, alpha_gamma_delta,
                 sigma_sq, j, k, l, i, m)

  dat <- data.frame(
    Y = Y,
    block = factor(i),
    source_credibility = factor(j),
    color = factor(k),
    headline_framing = factor(l)
  )

  fit <- lm(Y ~ source_credibility * color * headline_framing + block, data =
dat)
  p_val <- anova(fit)["source_credibility", "Pr(>F)"]
  return(as.integer(p_val < 0.05))
}
```

```

}

# Run 1000 simulations

n_sim <- 1000
results <- replicate(n_sim, one_sim(mu, alpha, gamma, delta, beta,
                                   alpha_gamma, alpha_delta, gamma_delta,
                                   alpha_gamma_delta,
                                   sigma_sq, i, j, k, l, m))

power <- mean(results)
power

```

```
[1] 0.782
```

```

#####
# Iterating over steps 1-3 multiple times (MC)
#####

# Number of simulations
iter <- 1000

# Run simulations
reject_vec <- replicate(
  iter,
  one_sim(mu, alpha, gamma, delta, beta,
          alpha_gamma, alpha_delta, gamma_delta, alpha_gamma_delta,
          sigma_sq, i, j, k, l, m)
)

# Estimated power
power_est <- mean(reject_vec)

power_est

```

```
[1] 0.783
```

```

library(ggplot2)

#####
# Running MC estimate of power
#####

running_power <- cumsum(reject_vec) / seq_along(reject_vec)

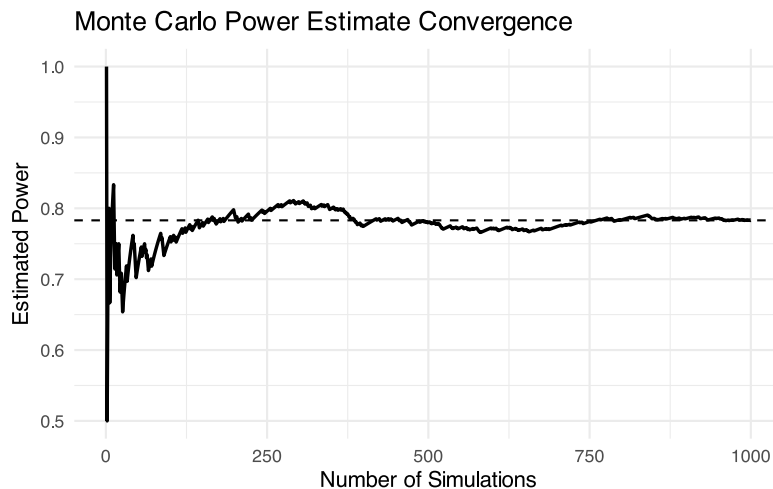
```

```

# Put into a data frame for ggplot
df_power <- data.frame(
  iter = seq_along(reject_vec),
  power = running_power
)

# Convergence plot
ggplot(df_power, aes(x = iter, y = power)) +
  geom_line(linewidth = 0.8) +
  geom_hline(yintercept = mean(reject_vec), linetype = "dashed") +
  labs(
    title = "Monte Carlo Power Estimate Convergence",
    x = "Number of Simulations",
    y = "Estimated Power"
  ) +
  theme_minimal()

```



```

#####
# Getting power as a function of the sample size
#####

sample_sizes <- c(1, 2, 5, 10, 20, 30, 50) # n per cell
n_sim <- 1000

power_by_n <- sapply(sample_sizes, function(n_per_cell) {

  # rebuild index vectors for this sample size
  n_total <- 16 * n_per_cell

  i <- rep(rep(1:2, each = 8), n_per_cell)
  j <- rep(rep(c(1,1,1,1,2,2,2,2), 2), n_per_cell)

```

```

k <- rep(rep(c(1,1,2,2,1,1,2,2), 2), n_per_cell)
l <- rep(rep(c(1,2,1,2,1,2,1,2), 2), n_per_cell)
m <- 1:n_total

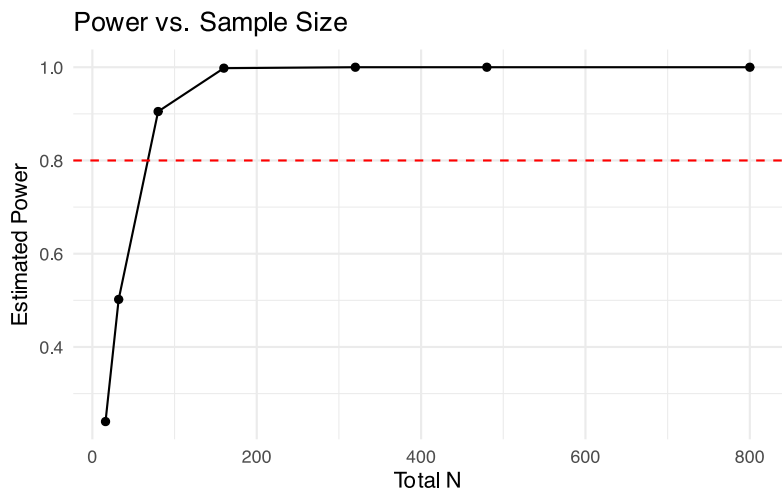
results <- replicate(n_sim, one_sim(mu, alpha, gamma, delta, beta,
                                     alpha_gamma, alpha_delta, gamma_delta,
alpha_gamma_delta,
                                     sigma_sq, i, j, k, l, m))

mean(results)
})

# Plot power curve
df_n <- data.frame(n_per_cell = sample_sizes, power = power_by_n)

ggplot(df_n, aes(x = n_per_cell * 16, y = power)) +
  geom_line() +
  geom_point() +
  geom_hline(yintercept = 0.8, linetype = "dashed", color = "red") +
  labs(
    title = "Power vs. Sample Size",
    x = "Total N",
    y = "Estimated Power"
  ) +
  theme_minimal()

```



```

df_n$total_n <- df_n$n_per_cell * 16
df_n

```

```

  n_per_cell power total_n
1           1 0.240      16

```

2	2	0.502	32
3	5	0.905	80
4	10	0.998	160
5	20	1.000	320
6	30	1.000	480
7	50	1.000	800

With a sample size of 50, we have a power of 0.72.

```
#####
# Finding the power of each factor/interaction by plugging these lines into
# the below function
#####

# swap this line for each factor/interaction:
p_val <- anova(fit)["source_credibility", "Pr(>F)"] # 0.798
p_val <- anova(fit)["color", "Pr(>F)"] # 0.429
p_val <- anova(fit)["headline_framing", "Pr(>F)"] # 0.715
p_val <- anova(fit)["source_credibility:color", "Pr(>F)"] # 0.085
p_val <- anova(fit)["source_credibility:headline_framing", "Pr(>F)"] # 0.07
p_val <- anova(fit)["color:headline_framing", "Pr(>F)"] # 0.072
p_val <- anova(fit)["source_credibility:color:headline_framing", "Pr(>F)"] #
0.053
```

```
#####
# Power Curves
#####

one_sim <- function(mu, alpha, gamma, delta, beta,
                    alpha_gamma, alpha_delta, gamma_delta, alpha_gamma_delta,
                    sigma_sq, i, j, k, l, m) {

  Y <- generate_Y(mu, alpha, gamma, delta, beta,
                 alpha_gamma, alpha_delta, gamma_delta, alpha_gamma_delta,
                 sigma_sq, j, k, l, i, m)

  dat <- data.frame(
    Y = Y,
    block = factor(i),
    source_credibility = factor(j),
    color = factor(k),
    headline_framing = factor(l)
  )

  fit <- lm(Y ~ source_credibility * color * headline_framing + block, data =
dat)
  p_val <- anova(fit)["source_credibility:color:headline_framing", "Pr(>F)"]
```

```

    return(as.integer(p_val < 0.05))
}

# Run 1000 simulations
n_sim <- 1000
results <- replicate(n_sim, one_sim(mu, alpha, gamma, delta, beta,
                                   alpha_gamma, alpha_delta, gamma_delta,
                                   alpha_gamma_delta,
                                   sigma_sq, i, j, k, l, m))
power <- mean(results)
power

```

```
[1] 0.052
```